

Выгрузка проекта из Replit на GitHub

Читать по порядку — шаг за шагом

Содержание

1. Р
еги.
2. С
стр
озд
ани
и в
а Но
вас
е не
ин с
заз
на д
изн
по з
ви
коф
ни е
ит,
маг
rep
она
оци
тор
ия

в пр
оек
те

7. Доб
8. авл
9. При
10. вяз
11. рас
12. ко
13. ро
14. от
15. та
16. ра
17. ре
18. по
19. ле
20. ко
21. до
22. со
23. ро
24. по
25. ро
26. ро
27. ро
28. ро
29. ро
30. ро
31. ро
32. ро
33. ро
34. ро
35. ро
36. ро
37. ро
38. ро
39. ро
40. ро
41. ро
42. ро
43. ро
44. ро
45. ро
46. ро
47. ро
48. ро
49. ро
50. ро
51. ро
52. ро
53. ро
54. ро
55. ро
56. ро
57. ро
58. ро
59. ро
60. ро
61. ро
62. ро
63. ро
64. ро
65. ро
66. ро
67. ро
68. ро
69. ро
70. ро
71. ро
72. ро
73. ро
74. ро
75. ро
76. ро
77. ро
78. ро
79. ро
80. ро
81. ро
82. ро
83. ро
84. ро
85. ро
86. ро
87. ро
88. ро
89. ро
90. ро
91. ро
92. ро
93. ро
94. ро
95. ро
96. ро
97. ро
98. ро
99. ро
100. ро

Зачем выгружать проект на GitHub?

GitHub — это облачное хранилище для кода. После выгрузки туда ваш проект будет:

- Доступен с любого компьютера в любой точке мира
- Защищён от случайного удаления или потери данных
- Готов к переносу на ваш собственный сервер одной командой `git clone`
- Иметь историю всех изменений — можно откатиться к любой версии

☒ Все действия на сайте GitHub выполняются в браузере (Chrome, Firefox, Safari). Действия в Replit — в специальной Shell-консоли внутри самого Replit.

Что нам понадобится

Что	Зачем
Аккаунт на GitHub	Для хранения вашего кода
Токен доступа (PAT)	GitHub-пароль для команд в консоли — обычный пароль не работает
Shell-консоль в Replit	Для выполнения команд Git
Имя и email	Git прикрепляет их к каждому сохранению кода

1

Регистрация аккаунта на GitHub

☒ Если у вас уже есть аккаунт на GitHub — переходите сразу к Шагу 2.

Откройте браузер и перейдите на сайт:

☒ <https://github.com>

На главной странице GitHub в правом верхнем углу нажмите кнопку:



Sign up

Большая зелёная кнопка или ссылка в правом верхнем углу

Заполните форму регистрации:

1. Enter your email — введите ваш email-адрес
2. Create a password — придумайте пароль (минимум 8 символов, должна быть хотя бы одна цифра и буква)
3. Enter a username — придумайте имя пользователя (только латинские буквы, цифры и дефис).
Например: `ivanov-facades`
4. Нажмите Continue
5. Пройдите верификацию (пазл или задача с картинками)
6. Нажмите Create account

На ваш email придёт письмо с кодом подтверждения. Введите код на странице GitHub и нажмите Enter.

Имя пользователя (username) будет частью адреса вашего репозитория: `github.com/ВАШ_USERNAME/facadebot` — выбирайте его внимательно.

2 Создание нового репозитория на GitHub

Репозиторий — это папка на GitHub, в которой будет храниться ваш проект.

После входа в аккаунт нажмите кнопку «+» в правом верхнем углу страницы:



New repository

Кнопка «+» → выпадающее меню → «New repository»

Откроется страница создания репозитория. Заполните поля:

1. Repository name — введите facadebot (только строчные буквы и дефисы)
2. Description (необязательно) — можете написать: «Telegram-бот для заказа фасадов»
3. Выберите видимость репозитория:

- **Private (рекомендуется)**

Репозиторий будет виден только вам. Правильный выбор для коммерческого проекта.

- Public

Репозиторий будет виден всем в интернете.

4. НЕ ставьте галочки на: «Add a README file», «Add .gitignore», «Choose a license» — мы создадим их сами
5. Нажмите зелёную кнопку:

Create repository

GitHub откроет страницу нового пустого репозитория. Вы увидите URL вашего репозитория вида:

`https://github.com/ВАШ_USERNAME/facadebot`

Скопируйте этот URL — он понадобится на Шаге 8. Адрес виден прямо в адресной строке браузера.

3 Открытие Shell-консоли в Replit

Консоль (Shell) — это текстовый интерфейс, где вы вводите команды. Именно здесь мы будем работать с Git.

В Replit Shell находится в нескольких местах:

Способ 1 — вкладка Shell (самый простой)

В правой части экрана Replit есть панель. Найдите вкладку с надписью:



Shell

Вкладка в правой панели, рядом с «Console» и «Output»

Нажмите на неё — откроется чёрный экран с мигающим курсором. Это Shell.

Способ 2 — через меню Tools

Если не видите вкладку Shell:

1. Нажмите на три полоски (☰) или «Tools» в верхнем меню Replit
2. В выпадающем списке выберите «Shell»

Вы увидите приглашение ввода, например:

```
$ shell
~/workspace $
```

Это значит, что Shell готов принимать команды. Всё, что мы будем вводить дальше — вводить именно здесь.

☑ Shell в Replit работает в корневой директории вашего проекта. Вам не нужно дополнительно переходить в папку.

4

Настройка Git — указываем имя и email

Git прикрепляет ваше имя и email к каждому сохранению (коммиту). Это нужно сделать один раз.

В Shell введите первую команду (замените «Ваше Имя» на своё имя и нажмите Enter):

```
$ shell
git config --global user.name "Ваше Имя"
```

Затем введите вторую команду (замените email на тот, что использовали при регистрации на GitHub):

```
$ shell
git config --global user.email "ваш@email.ru"
```

Проверьте, что данные сохранились:

```
$ shell
git config --global --list
```

В ответ Shell выведет что-то вроде:

```
$ shell
```

```
# Ожидаемый результат:
```

```
user.name=Ваше Имя
```

```
user.email=ваш@email.ru
```

❏ Эти данные нигде не отображаются публично в приватном репозитории. Они нужны только для истории изменений.

5

Инициализация Git-репозитория в проекте

Git-репозиторий — это невидимая папка `.git` внутри проекта, которая отслеживает все изменения файлов.

Сначала убедимся, что мы находимся в правильной папке проекта:

```
$ shell
```

```
pwd
```

Ответ должен быть:

```
$ shell
```

```
# Ожидается примерно:
```

```
/home/runner/workspace
```

Теперь инициализируем репозиторий:

```
$ shell
```

```
git init
```

Shell ответит:

```
$ shell
```

```
# Ожидаемый результат:
```

```
Initialized empty Git repository in /home/runner/workspace/.git/
```

❏ Если папка `.git` уже существует (Git уже был инициализирован ранее через интерфейс Replit), команда `git init` сообщит об этом — это нормально, продолжайте.

6

Создание файла `.gitignore`

`.gitignore` — это список файлов и папок, которые НЕ нужно отправлять на GitHub. В него входят: пароли из `.env`, временные файлы, папки с зависимостями `node_modules`.

Проверьте, существует ли файл `.gitignore`:

```
$ shell
cat .gitignore
```

Возможны два варианта:

Вариант А — файл уже существует

Shell покажет его содержимое. Убедитесь, что в нём есть следующие строки. Если каких-то нет — добавьте командой:

```
$ shell
echo '.env' >> .gitignore
echo 'node_modules/' >> .gitignore
echo 'dist/' >> .gitignore
echo '.local/' >> .gitignore
echo '*.log' >> .gitignore
```

Вариант Б — файла нет (ошибка: No such file or directory)

Создайте `.gitignore` одной командой:

```
$ shell
cat > .gitignore << 'EOF'
.env
node_modules/
dist/
.local/
*.log
.replit
replit.nix
.breakpoints
EOF
```

Проверьте результат:

```
$ shell
cat .gitignore
```

Должны увидеть список строк, которые указали выше.

❗ **ВАЖНО:** Строка `.env` обязательна. Без неё файл с паролями и токенами попадёт на GitHub — это серьёзная угроза безопасности.

7

Добавление файлов и первый коммит

Коммит — это «моментальный снимок» состояния проекта. Первый коммит — самый важный.

7.1 — Посмотрим, какие файлы Git видит

```
$ shell
git status
```

Shell покажет список файлов. Красным — не отслеживаемые, зелёным — добавленные. Файлы из `.gitignore` не будут показаны.

7.2 — Добавляем все файлы в очередь на отправку

```
$ shell
git add .
```

Точка (.) означает «добавить все файлы из текущей папки и подпапок». После этой команды Shell ничего не ответит — это нормально.

7.3 — Проверяем, что добавилось

```
$ shell
git status
```

Теперь все файлы должны быть зелёного цвета — это означает, что они готовы к коммиту.

7.4 — Создаём первый коммит

```
$ shell
git commit -m "Первая выгрузка проекта FacadeBot"
```

Shell ответит примерно следующее:

```
$ shell
# Ожидаемый результат:
[main (root-commit) abc1234] Первая выгрузка проекта FacadeBot
47 files changed, 3821 insertions(+)
create mode 100644 package.json
...
```

📌 Текст после `-m` в кавычках — это описание коммита. Пишите понятные описания: «Добавлена генерация счетов», «Исправлена ошибка отправки email» и т.д.

8

Привязка репозитория GitHub к проекту

Теперь нужно сказать локальному Git, куда отправлять файлы. URL репозитория вы скопировали на Шаге 2.

Введите команду (замените `ВАШ_USERNAME` на ваш логин GitHub):

```
$ shell
git remote add origin https://github.com/ВАШ_USERNAME/facadebot.git
```

Команда не выводит ничего в ответ — это нормально. Проверьте, что адрес сохранился:

```
$ shell
git remote -v

# Ожидаемый результат:
origin https://github.com/ВАШ_USERNAME/facadebot.git (fetch)
origin https://github.com/ВАШ_USERNAME/facadebot.git (push)
```

Если адрес оказался неверным (опечатка), исправьте его:

```
$ shell
git remote set-url origin https://github.com/ВАШ_USERNAME/facadebot.git
```

9

Создание и настройка токена доступа (Personal Access Token)

GitHub больше не принимает обычный пароль при работе через консоль. Вместо этого нужен специальный токен — Personal Access Token (PAT).

9.1 — Переходим в настройки GitHub

В браузере войдите в GitHub. В правом верхнем углу нажмите на аватар (фото/иконку профиля):

 **Ваш аватар/иконка профиля**
Правый верхний угол сайта GitHub

В выпадающем меню выберите:

 **Settings**
Настройки аккаунта — предпоследний пункт в меню

9.2 — Переходим к токенам

В левом меню настроек прокрутите вниз до раздела:

 **Developer settings**
Самый нижний пункт левого меню

На открывшейся странице нажмите:

 **Personal access tokens**
В левом меню «Developer settings»

Выберите:

 **Tokens (classic)**
Классические токены — самый простой вариант

Нажмите кнопку:

Generate new token

В подменю выберите:

 **Generate new token (classic)**

9.3 — Заполняем параметры токена

1. Note (название) — введите: replit-facadebot
2. Expiration (срок действия) — выберите: No expiration (без срока) или 1 год
3. Select scopes (права доступа) — поставьте галочку ТОЛЬКО на:

 **repo** — полный доступ к репозиториям

4. Нажмите зелёную кнопку:

Generate token

9.4 — Сохраняем токен

GitHub ОДИН РАЗ покажет ваш токен — длинную строку символов, начинающуюся на ghp_...

Скопируйте токен и сохраните в надёжном месте прямо сейчас!

Нажмите кнопку с иконкой копирования рядом с токеном. Вставьте в текстовый файл или запишите.

 После закрытия страницы токен больше НЕ будет показан. Если потеряли — придётся создать новый (это не страшно, просто повторите шаги 9.1–9.4).

9.5 — Сохраняем токен в Replit (чтобы не вводить каждый раз)

Вернитесь в Shell в Replit и выполните команду (замените ВАШ_USERNAME и TOKEN):

```
$ shell
git config --global credential.helper store
# После этой команды при первом git push Git запросит логин и пароль.
# Логин: ваш GitHub username
# Пароль: вставьте токен (не обычный пароль!)
```

 Токен вводится как пароль при первом git push (Шаг 10). После этого Git запомнит его и больше не будет спрашивать.

10 Отправка проекта на GitHub (git push)

Это финальный шаг — отправляем все файлы в репозиторий GitHub.

Введите команду:

```
$ shell
git push -u origin main
```

Что может произойти:

Сценарий А — Git запрашивает логин и пароль

Shell покажет:

```
$ shell
Username for 'https://github.com': БАШ_USERNAME
Password for 'https://БАШ_USERNAME@github.com':
```

Введите:

1. Username — ваш GitHub username (логин)
2. Password — вставьте ТОКЕН из Шага 9 (не обычный пароль!)

🔒 При вводе токена символы не отображаются в консоли — это нормально, просто вставьте и нажмите Enter.

Сценарий Б — успешная отправка

Shell покажет что-то вроде:

```
$ shell
# Ожидаемый результат:
Enumerating objects: 89, done.
Counting objects: 100% (89/89), done.
Delta compression using up to 8 threads
Compressing objects: 100% (72/72), done.
Writing objects: 100% (89/89), 234.56 KiB | 1.23 MiB/s, done.
Total 89 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/БАШ_USERNAME/facadebot.git
* [new branch] main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

🔒 Это означает успех. Ваш проект теперь на GitHub!

Сценарий В — ошибка «src refspec main does not match any»

Это значит, что Git использует ветку master вместо main. Исправляется так:

```
$ shell
git branch -M main
git push -u origin main
```

Сценарий Г — ошибка «Authentication failed»

Неверный токен или username. Попробуйте:

```
$ shell
git config --global --unset credential.helper
git push -u origin main
# Введите username и токен заново
```

11 Проверка — файлы появились на GitHub

Откройте браузер и перейдите на страницу вашего репозитория:

```
https://github.com/БАШ_USERNAME/facadebot
```

Вы должны увидеть:

- Список папок и файлов вашего проекта
- Описание последнего коммита рядом с каждым файлом
- Дату и время последнего изменения

Проверьте, что секретные файлы НЕ попали на GitHub:

- Файла .env быть НЕ должно
- Папки node_modules/ быть НЕ должно
- Папки dist/ быть НЕ должно

❗ Если вы видите файл .env в репозитории — немедленно удалите его (инструкция ниже) и смените все пароли и токены из этого файла.

Как удалить случайно загруженный .env

```
$ shell
git rm --cached .env
git commit -m "Удалён .env из репозитория"
git push
```

12 Как обновлять проект после изменений

После каждого значительного изменения в проекте выполняйте три команды в Shell:

Команда	Что делает
<code>git add .</code>	Добавить все изменённые файлы
<code>git commit -m "..."</code>	Создать коммит с описанием изменений
<code>git push</code>	Отправить на GitHub

Пример полного цикла обновления после доработки проекта:

```
$ shell
```

```
# 1. Добавить все изменения
```

```
git add .
```

```
# 2. Создать коммит с понятным описанием
```

```
git commit -m "Добавлен новый регион — Краснодарский край"
```

```
# 3. Отправить на GitHub
```

```
git push
```

❏ После первого `git push -u origin main` (Шаг 10) последующие обновления отправляются просто командой `git push` — без дополнительных параметров.

Полезные дополнительные команды

Команда	Что делает
<code>git status</code>	Показывает изменённые файлы
<code>git log --oneline</code>	История коммитов в сжатом виде
<code>git diff</code>	Показывает, что именно изменилось
<code>git pull</code>	Скачать изменения с GitHub (если редактировали там)

Итоговый чеклист

- ❏ Зарегистрирован аккаунт на GitHub
- ❏ Создан приватный репозиторий `facadebot`
- ❏ Открыта Shell-консоль в Replit
- ❏ Настроены имя и email в Git (`git config --global`)
- ❏ Выполнена инициализация: `git init`
- ❏ Создан файл `.gitignore` со строкой `.env`
- ❏ Добавлены файлы: `git add .`
- ❏ Создан первый коммит: `git commit -m "..."`
- ❏ Привязан URL репозитория: `git remote add origin ...`
- ❏ Создан Personal Access Token на GitHub (раздел Settings ❏ Developer settings)
- ❏ Проект отправлен на GitHub: `git push -u origin main`
- ❏ В браузере на GitHub видны файлы проекта
- ❏ Файла `.env` и папки `node_modules/` HET в репозитории

Следующий шаг — перенос на сервер

После выгрузки на GitHub проект можно перенести на Ubuntu-сервер одной командой:

```
git clone https://github.com/BAШ_USERNAME/facadebot.git
```

Подробная инструкция по настройке сервера — в документе «FacadeBot: Руководство по развёртыванию на Ubuntu Server».

